

EF Core 8.0 Guided Hands-On Exercises — Answer Key

Lab 4: Inserting Initial Data into the Database

Scenario	The store manager wants to add initial product categories and products to the system.
Objective	Use EF Core to insert records using <code>AddAsync</code> and <code>SaveChangesAsync</code> .

Step 1: Insert Data in Program.cs

```
using var context = new AppDbContext();

var electronics = new Category { Name = "Electronics" };
var groceries   = new Category { Name = "Groceries" };

await context.Categories.AddRangeAsync(electronics, groceries);

var product1 = new Product { Name = "Laptop",    Price = 75000, Category = electronics };
var product2 = new Product { Name = "Rice Bag",  Price = 1200,  Category = groceries };

await context.Products.AddRangeAsync(product1, product2);

await context.SaveChangesAsync();
```

Explanation:

- **AddRangeAsync** stages multiple entities for insertion in one call; it marks their state as *Added* in the change tracker but does not touch the database yet.
- Setting **Category = electronics** on each product (instead of manually setting `CategoryId`) lets EF Core's change tracker automatically resolve the foreign key once the category is saved and gets its generated Id — this is called **relationship fixup**.
- **SaveChangesAsync()** is what actually opens a connection and executes the SQL. EF Core wraps all pending inserts in a single transaction, inserts Categories first (parents), then Products (children), so the foreign key constraint is satisfied automatically.

Step 2: Run the App

```
dotnet run
```

Executes Program.cs top to bottom. On first run this inserts the two categories and two products into the database.

Step 3: Verify in SQL Server

In SSMS/Azure Data Studio, run:

```
SELECT * FROM Categories;
SELECT * FROM Products;
```

You should see 2 rows in Categories (Electronics, Groceries) and 2 rows in Products (Laptop linked to Electronics's Id, Rice Bag linked to Groceries's Id).

Good Practice Note

Running this Program.cs a second time will insert duplicate rows, since there's no check for existing data. For repeatable seeding, guard with a check such as `if ( !context.Categories.Any() )` before inserting, or use the `HasData()` migration-based seeding approach covered later in Lab 9.

Key Takeaway

`AddRangeAsync` + `SaveChangesAsync` is the standard EF Core pattern for writing data: stage changes in memory, then commit them to the database in one transactional call.